

HammerSim: A System-Level Tool to Model RowHammer

Kaustav Goswami, Ayaz Akram, Hari Venugopalan, and Jason Lowe-Power

University of California, Davis

Abstract

Modern architecture research relies on simulators to evaluate system security, yet analyzing emerging hardware vulnerabilities like RowHammer requires full-system visibility. As RowHammer vulnerabilities worsen with continuous technology scaling, existing simulators lack the system-level models needed to study complex OS effects and cross-layer mitigations. This tool deficiency leaves modern computing platforms exposed to severe reliability and security risks. In this work, we present HammerSim, a gem5-based framework for modeling RowHammer at the full-system level. HammerSim integrates probability-driven bitflip modeling to realistically capture the behavior of RowHammer. It further enables evaluation of hardware and software mitigations such as TRR and selective ECC. We validate HammerSim’s bitflip modeling against real DDR4 DIMMs using JS divergence, demonstrating its utility in studying attacks, defenses, and benign workload susceptibility. Our framework provides an extensible platform to bridge the gap between hardware experiments and architectural simulation.

1. Introduction

Dynamic random access memory, or DRAM, is the *de-facto* choice for main memory design due to its cost-effectiveness. However, due to scaling, researchers have noticed variations in the nominal parameters of these devices [9, 100, 51]. Recently, it has been found that accessing (or ACTivating) the same cell or the same set of cells on a DRAM module repeatedly causes data corruption in the neighboring cells. This is called RowHammer [5, 44]. Researchers have shown several system-level attacks using RowHammer to escalate privileges, fingerprint or leak private data [48, 93, 83, 8, 84, 68, 27, 91, 3, 10, 13, 12, 19, 20, 26, 35].

Currently, researchers rely on two primary hardware-based methods to study RowHammer: (a) FPGAs, and (b) software-based exploits [70, 69, 88, 1]. FPGA-based memory controllers [31, 64, 66] (SoftMC [31], DRAM-Bender [66], *etc.*) provide highly accurate timing and have been instrumental in uncovering proprietary in-DRAM mitigations like Target Row Refresh (TRR) [30]. However, FPGAs lack the capability to model OS-level abstractions or full server-class system dynamics. Conversely, software-based RowHammer exploits [20, 36, 14, 91, 41, 27, 92] successfully bypass OS-level protections and allow researchers to explore kernel-level vulnerabilities [3, 97]. Yet, they do not provide the hardware visibility necessary to prototype or evaluate new hardware-software co-design mitigations.

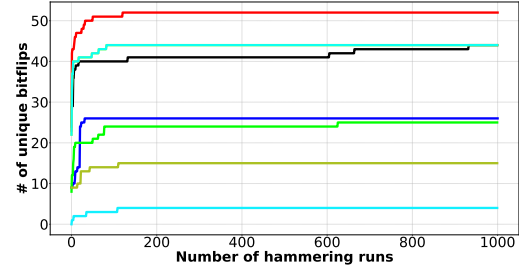


Figure 1: Count of unique bitflips on different DRAM DIMMs’ row. Each line plot corresponds to a victim row. In total, we collected victim row data from 9 different DIMMs from two different manufacturers.

Researchers today use cycle-level simulations [44, 98, 87, 71, 47, 25, 85] to bridge this gap between the hardware-level and software-level view of RowHammer. However, popular cycle-accurate and cycle-level simulators [79, 49, 45, 53, 75] used for memory system research today assume an idealistic setting, often using timing, current, and voltage values from data sheets for simulating hardware components [94, 79, 49, 45, 75, 53]. To model variations, researchers have to explicitly add variations [22, 100, 86], which generally leads to over/under-estimation of the simulation model. While some models of process variation exist purely as a statistical models [82, 2, 86], we do not have models for RowHammer. Further, the RowHammer threshold is decreasing as DRAMs are becoming denser [43], which motivates the need for such a model.

To understand the fidelity gap of such uniform models, we performed extensive RowHammer characterization experiments across multiple real-world DRAM Dual In-line Memory Modules (DIMMs). As shown in Figure 1, our hardware evaluations reveal that RowHammer-induced bitflips are highly non-uniform; unique bitflips saturate over time, and certain memory regions exhibit significantly higher vulnerability than others.

Crucially, prior literature demonstrates that real-world RowHammer behavior is deeply intertwined with full-system parameters, including the operating system page allocation [10, 84, 36], CPU frequency [93], cache coherency protocols [51], instruction set architectures (ISAs) [44, 84, 20, 36, 3, 92, 56], and underlying memory technology [44, 47, 30]. Even benign applications running under normal system workloads have been shown to inadvertently trigger RowHammer bitflips [67]. Without the ability to simulate

how hardware faults dynamically interact with OS scheduling and cache hierarchies, researchers cannot reliably estimate security boundaries or system reliability.

To solve this, we propose HammerSim, a full-system RowHammer modeling framework built upon the gem5 architectural simulator [53]. HammerSim directly addresses the shortcomings of prior tools by combining the physical accuracy of hardware-characterization maps with the system-level visibility of an architectural simulator. While circuit-level models [40] or trace-driven simulators cannot natively capture live OS reactions, HammerSim runs an unmodified OS. It enables researchers to analyze realistic RowHammer data corruption under complex workloads, cache hierarchies, and OS scheduling, while actively supporting the evaluation of sophisticated hardware-software mitigation co-designs

Overall, the contributions of this paper are as follows:

- System-level RowHammer model in gem5 that incorporates multiple probability distributions for realistic bitflip estimation, enabling studies on real workloads and future DRAM technologies with or without mitigations.
- Dual analysis modes with data corruption support where an *online mode* enables in-situ data corruption during simulation to study system-level effects, and an *offline mode* provides post-processing for accurate probability estimation and correction of over/under-estimations.
- Hardware-informed modeling and validation where extensive RowHammer experiments on DDR4 DIMMs to capture real-world bitflip patterns and validate simulation fidelity using JS divergence.

The rest of the paper is organized as follows. Section 2 discusses the basics of a DRAM device, the RowHammer vulnerability and Section 3 describes our RowHammer model in a holistic manner. Section 5 analyzes the simulations. The work is concluded in Section 7 with further scope of improvement.

2. Background

In this section, we introduce the basics of a DRAM device, how it is implemented as an interface in gem5, and the RowHammer vulnerability.

2.1. DRAM Organization

DRAM has spawned several generations and technologies. This includes DDRx [59, 60, 62, 39], GDDRx [33, 63], LPDDRx [58], DDRxL [61], etc. Each physical DRAM Dual Inline Memory Module or DRAM DIMM is installed on a DRAM *channel* on the motherboard. Channels permit issuing concurrent memory requests to multiple DIMMs. A DIMM contains multiple logical structures called *banks* on one or both of the *ranks*. A bank is a two-dimensional array of cells organized into *rows* and *columns*. Each cell contains a capacitor and an access transistor, with the capacitor's charged state representing a single bit. Organizing cells into banks enables faster memory access since memory addresses

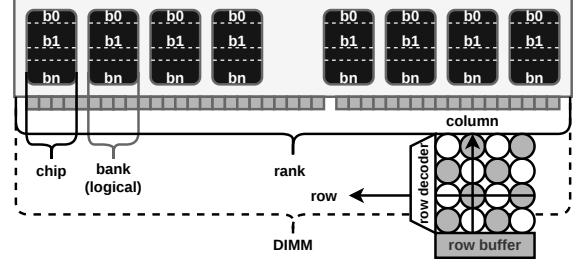


Figure 2: This figure shows a single rank of a DRAM DIMM. Each rank contains multiple logical structures called banks that are interspersed across multiple physical structures called chips.

that map to different banks can be accessed in parallel. Independent dies are packaged into multiple physical entities called *chips*. The banks on each rank are uniformly distributed across these chips (Figure 2).

2.2. Working of a DRAM Device

The memory controller receives memory requests with an instruction and a corresponding address to the DRAM. These addresses encode information about the DRAM structure (channel, rank, bank, etc). A mapping function uniquely maps addresses to a set of capacitors within the DRAM [73]¹. On receiving a memory request, the memory controller uses this function to identify the corresponding bank within the corresponding DIMM. The address is then pushed into the bank's *row-decoder* logic. Then the row-decoder selects the appropriate row and transfers it to a *row-buffer*, which is known as an *ACTIVATE*, or *ACT*. The row-buffer is an array of sense amplifiers that sense the charge of the cells in that row to determine their bit values. Finally, the bank's *column-decoder* reads the values of the appropriate cells within the row-buffer and returns the corresponding data.

DRAM technology is based on capacitors, which inherently lose its charge over time [34]. Their contents have to be recharged periodically to preserve the integrity of the data stored. The memory controller issues a *REFRESH* (*REFI*) instruction to a subset of rows every 7.8 μ s (*tREFI*) to preserve the values of their capacitors. To refresh a particular row, it is first activated to sense the bit values. Then, the bits are subsequently restored to the DRAM array. Every DRAM capacitor gets refreshed at least once every 32 ms or 64 ms (*tREFW*) based on the operating temperature.

2.3. gem5's Memory Controller and Memory Interface

The memory controller in gem5 is simulated as an event-driven component [28]. It models a generic memory

1. Note that this is not a virtual-to-physical address mapping. The virtual address is translated to its corresponding physical address in the Translation Lookaside Buffer (TLB) [72]. This mapping is physical to device mapping.

controller with split queues for reads and writes, and a shared queue for responses. It is responsible for managing the communication between the processor and the memory. gem5 receives memory requests from the processor and schedules these requests based on a given scheduling policy. The memory organization it models is similar to its regular counterpart. Addresses decode into rank, bank, row, and column at each memory controller. The channel interleaving is decoded outside at a separate crossbar. A Packet is used to encapsulate a transfer between two objects in the memory system [53]. An LLC miss in gem5 is seen as a packet coming into the memory controller. Once the packet is received at the memory controller, it decodes that packet, schedules it, and sends it to the memory interface.

Memory devices are modeled as classes in gem5. Currently in gem5, there are two memory interfaces: (a) DRAM and (b) NVM. The timing parameters for these devices are based on the technical sheet of these devices. Hansson *et al.* [28] mentions that gem5 does not model all the timing parameters from a contemporary memory device. However, gem5 models all the important timing parameters, which were validated against a cycle-accurate DRAM simulator.

2.4. The RowHammer Vulnerability

Scaling of DRAM devices over the generations has resulted in the deviation of their nominal parameters from their theoretical counterpart due to variations [100, 9, 83]. Modern DIMMs are susceptible to memory corruption as a result of electrical interference among the cells [54]. One such technique to leverage bit flips without accessing the row is the rowhammer (RH) exploit [5, 44]. It causes memory corruption by repeatedly accessing two addresses that map to two different rows in a single bank. This leads to the repeated activation and deactivation of the two rows back-and-forth between the row-buffer and the DRAM bank array. The resulting electro-magnetic interference between the accessed rows (referred to as *aggressor rows* henceforth) and their neighboring rows (referred to as *victim rows* henceforth) accelerates the rate at which the capacitors in the victim rows lose their charge. Once capacitors have lost a sufficient amount of charge, refreshing the DRAM does not restore their value, resulting in memory corruption in the form of bit flips.

There have been several works done in the past to detect and mitigate RH attacks [44, 87, 85, 47, 71, 98]. DRAM manufacturers implement TRR added the standard mitigation mechanism in DDR4 devices as a practical solution that does not violate DRAM timings [81, 62, 20].

3. HammerSim

Our objective is to develop a comprehensive, full-system simulation framework that accurately mirrors the hardware-level probabilistic and spatial characteristics of RowHammer. In this section, we list the challenges, perform extensive hardware exploration, incorporate the findings and discuss the implementation details.

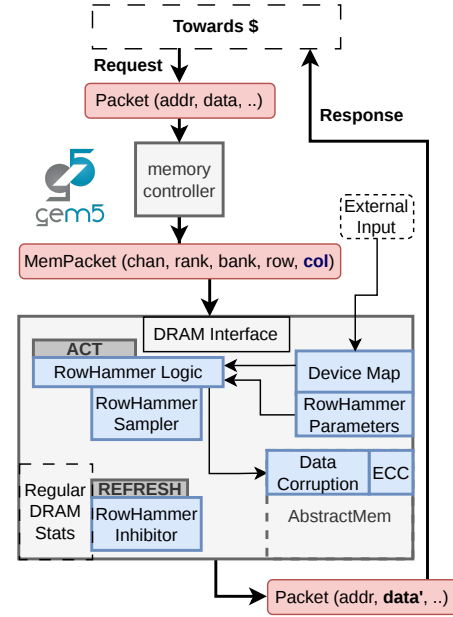


Figure 3: High-level overview of RowHammer Modeling in gem5. HammerSim architectural components are highlighted in blue. The core modeling logic is integrated within the gem5 DRAM interface, while the execution path is configured via flexible user parameters.

3.1. Architectural and Simulation Challenges

Standard circuit-level or trace-driven memory simulators cannot resolve the three fundamental design challenges introduced when designing a scalable, full-system simulation infrastructure for RowHammer:

- C1 Representing Non-Uniform Hardware Non-Idealities.** Traditional architectural simulators treat faults as uniform or binary events. However, real-world hardware data reveals that RowHammer bitflips are heavily non-uniform and highly localized due to manufacturing process variations. Translating these irregular physical vulnerabilities into deterministic, flexible, and mathematically sound probability models within an architectural simulator is highly non-trivial.
- C2 Spatial Tracking Overhead at Scale.** To model multi-row vulnerabilities (e.g., Half-Double [23, 46] or multi-sided attacks [44, 20]), the memory controller must continuously maintain and update activation tracking structures across spatial neighborhoods. Performing these multi-row spatial lookups at runtime for every memory access introduces severe simulation timing and memory overheads, requiring highly optimized meta-data tracking.
- C3 Balancing Execution Fidelity with Multi-Parameter Exploration.** To evaluate hardware-software defenses,

TABLE 1: Experimental System Parameters

Parameter	Specification
CPU type	Intel(R) Core(TM) i7-7700 CPU
CPU core count	4
Frequency	3.6 GHz
L1 cache per core	32 KiB + 32 KiB (I + D)
L2 cache	1 MiB
L3 cache	8 MiB
DRAM manufacturers	SK Hynix, Samsung
DRAM technology	DDR4
DRAM size	8 GB

researchers require an *online mode* capable of dynamically corrupting live memory data to capture operating system reactions and application-level crashes. Conversely, security researchers exploring different hardware topologies require rapid, trace-driven sensitivity sweeps. The framework must seamlessly decouple full-system execution from trace-driven analytical processing without duplicating simulator infrastructure.

3.2. Hardware Characterization Experiments

To ground our architectural probability models in physical ground truth, we conducted extensive hardware characterization experiments across multiple commercially available DDR4 DIMMs. The goal of these experiments is to quantify the exact spatial and temporal probability distributions governing RowHammer failures under controlled environments.

We deployed the Blacksmith [36, 37] profiling infrastructure on a desktop detailed in Table 1. We executed double-sided RowHammer profiling by targeting a specific victim row sandwiched between two continuously activated aggressor rows. Each profiling loop executed 5×10^6 ACTs, and the routine was repeated over 1,000 distinct iterations to observe long-term bitflip patterns, cumulative error saturation, and localized vulnerability profiles.

3.3. Empirical Observations

Our physical characterization yielded three core insights regarding the nature of modern DRAM vulnerabilities, which serve as the direct mathematical foundations for the HammerSim architectural abstraction layers:

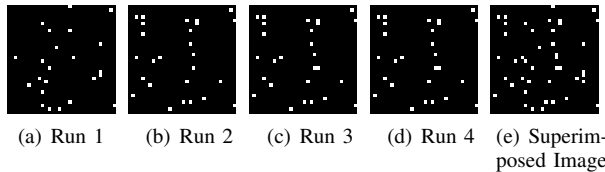


Figure 4: Observed bitflips on a single row on a width 8 DRAM DIMM (row 1278). There are 1024 columns per row. Each of these images presents a 32x32 shaped 2D image of a single row. A white dot represents a flipped column.

Observation 1 (Spatial Localization): RowHammer-induced bitflips are intensely localized within the memory array. While specific regions of a given DRAM row are highly vulnerable to RowHammer, vast adjacent structures remain entirely resilient.

This behavior (Figure 4) indicates that local manufacturing process variations dictate cell weakness. Rather than hard-coding these patterns for a single device, HammerSim resolves *C1* by providing two generalized modeling paths: an ingested, file-based physical device map, or a statistical process variation engine based on VARIUS [82] that generates multivariate normal distributions across the simulated memory coordinates to flag distinct *Weak Cells* (WC).

Observation 2 (Temporal Saturation): The unique bitflip count across a DRAM array saturates over time. Cells that fail during an initial activation cycle demonstrate a bounded, distinct probability p of failing again across subsequent cycles.

The empirical histograms in Figure 5 shows this saturation behavior. This establishes that the pool of vulnerable cells is structurally finite. Once a simulated cell is identified as a WC, HammerSim evaluates a uniform probability distribution at runtime to determine if an active activation stresses the cell into a flippable state (F_{WC}). We measured this baseline bitflip probability between $\frac{1}{5 \times 10^{10}}$ and $\frac{1}{5 \times 10^8}$ per activation, which we expose as a user-tunable parameter.

Observation 3 (Attack Proximity Non-Linearity): The probability of inducing a data failure scales non-linearly by multiple orders of magnitude when executing coordinated multi-sided RowHammer patterns relative to single-sided access sequences.

Our physical tracking demonstrated that executing a double-sided RowHammer sequence amplifies the bitflip probability by a factor of 2.5×10^3 compared to a single-

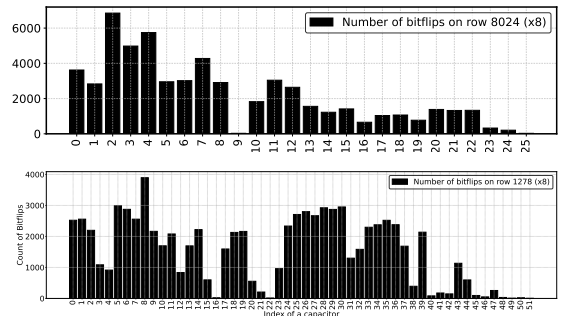


Figure 5: Histogram of bitflip counts on row number 8024 (top) and 1278 (bottom) across 1000 runs.

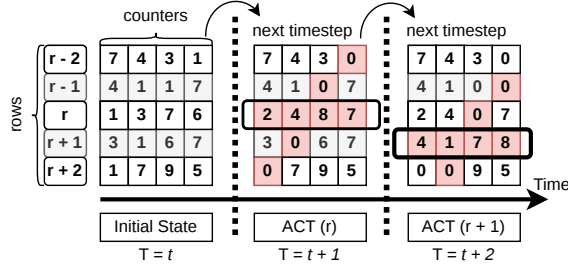


Figure 6: HammerSim maintains structural counter-pairs per DRAM row to dynamically track multi-row RowHammer access patterns. All counters are reset at each tREFW boundary. This figure shows 3 states of these counters: initial, after reading row r and after reading row $r + 1$.

sided baseline hitting the identical weak cell cluster. To implement this insight, HammerSim leverages dynamic tracking counters inside the memory interface, tracking the non-linear threat scaling of single-sided, double-sided, and generalized multi-row patterns.

3.4. Architectural Implementation

To incorporate these empirical insights into a unified simulation framework, HammerSim introduces structural and systemic modifications to the gem5 DRAM interface, implementing both online and offline execution modalities. In Appendix A, we provide details on navigating the HammerSim source-code.

3.4.1. Counter-Pairs and Adjacency Tracking. To resolve C2 and capture complex spatial interactions without exponential simulation overheads, HammerSim maintains optimized tracking structures within the memory controller. As illustrated in Figure 6, each row tracks its immediate neighborhood using counter-pairs. For standard vulnerabilities, this mechanism monitors single and double-sided access rates.

To support advanced vulnerabilities like Half-Double exploits [23], we expand the tracking radius to monitor four dedicated row activation counters ($r \pm 1, r \pm 2$) relative to the target row r . To optimize performance, these structures evaluate fault triggers during standard refresh intervals (tREFW), clearing all tracking metadata at each window boundary.

3.4.2. Online Mode: Runtime Fault Injection and Functional ECC. In *online mode*, HammerSim couples its hardware-informed probability distributions directly into gem5’s operational memory execution path. Every memory request dynamically triggers fault evaluation checks based on current row activation states, injecting live, bit-level data corruption into the simulator’s active memory arrays.

Because physical RowHammer bitflips occur silently within the DRAM capacitors without throwing hardware interrupts, they act as transient soft errors. HammerSim replicates this behavior: while all injected errors are logged

to standard output (stdout) for diagnostic tracking, the simulated application and operating system remain unaware of the corruption until they actively address or validate the corrupted memory address. This allows for authentic evaluation of silent data corruption across OS-level page schedulers, security kernel modules, and user space applications.

To support the exploration of cross-layer resilience, we implement a functional Error Correcting Code (ECC) abstraction layer within gem5’s memory subsystem. ECC-enabled DIMMs [89] are a popular option for server-class systems [13]. RowHammer on ECC-enabled DIMMs is a popular topic of research for both attacks [13] and defenses [55, 96, 77].

We implement SECDED algorithm from the reverse-engineered paper by Cojocar *et al.* [13]. HammerSim pads every word with parity blocks calculated via a user-defined text evaluation matrix (pMatrix). At runtime, when a corrupted row is accessed, the ECC module intercepts the request, evaluates the parity matrix, and dynamically updates native gem5 telemetry statistics (stats.txt) to record correctable single-bit faults or fatal, uncorrectable multi-bit errors.

3.4.3. Offline Mode: Decoupled Trace Analysis. To address C3 and enable rapid design space exploration without the penalty of repetitive, long-running full-system simulations, HammerSim introduces an optimized *offline mode*. When active, the memory controller logs row-level activation (ACT) allocations to a structured trace file at every tREFW boundary.

An external analytical tool subsequently processes the RowHammer trace that overlays the workload’s spatial activation footprint against alternative process variation distributions, device maps, or mitigation thresholds. This decoupling allows hardware designers and security analysts to evaluate how an identical application execution profile interacts with dozens of distinct DRAM manufacturing variations instantly, eliminating the need to re-run the underlying architectural simulation.

In Appendix B, we list all the available HammerSim simulation parameters introduced.

4. Discussion and Limitations

4.1. RowHammer Research Enabled

HammerSim allows researchers to investigate RowHammer attacks, defenses, and, also potentially study applications of RowHammer. In this section, we discuss several use cases that HammerSim enables.

4.1.1. RowHammer Probability Distribution Study. HammerSim allows researchers to investigate different probability distributions of causing a bitflip. We incorporate known probabilities from prior works [93, 44, 20, 83, 23, 46] and our experimental observations. However, there is no mathematical evidence that these are all the distributions that define RowHammer. The probability of a bitflip of a

strong DRAM cell, for example, is still missing from our evaluation. HammerSim can be extended in the future to incorporate such distributions, when discovered, and studied in a full-system environment.

4.1.2. RowHammer Attack. HammerSim allows users to uncover sophisticated memory access patterns that cause RowHammer attacks. Creating a RowHammer attack as a memory trace is straightforward by using gem5 traffic generators. Such traffic generators produce deterministic access patterns for a given device map. Various RowHammer attacks like Juggernaut attack pattern [95], double RowHammer attack [25], *etc.*, can be created using HammerSim. These maps can be refined by researchers or vendors using process variation data.

In contrast, implementing RowHammer at the system level is far more complex. Prior attacks [10, 84, 3, 20, 36, 41, 14, 27] show that bypassing OS abstractions is challenging. While evicting caches suffices for single-sided RowHammer on x86 systems, obtaining contiguous rows on the same bank in multi-channel, multi-rank servers remains difficult [93]. Furthermore, physical address mapping strongly influences RowHammer bitflip behavior [73, 20].

Further, HammerSim can model GPU-based RowHammer attacks [92, 50, 74, 32] using the GPU model in gem5 [76, 78].

4.1.3. RowHammer Mitigation. HammerSim enables users to prototype and evaluate hardware-based RowHammer mitigations at the system level. It supports two key components: (a) detection and (b) inhibition, modeled with timing correctness. To demonstrate this flexibility, HammerSim replicates simplified TRR mechanisms based on reverse-engineered designs [30, 36] from major DRAM vendors, including counter-based and probabilistic refresh strategies. These implementations show how researchers can extend HammerSim’s data structures to explore new mitigation techniques without breaking DRAM timing constraints.

TRR refreshes are sent out when the DRAM DIMM is locked (for tRFC time) during refreshes (via REFI instructions). Most research-based RowHammer mitigations, on the other hand, explicitly send out refreshes as ACTIVATE to the victim rows, violating DRAM timings. DRAM devices adhere to strict timing requirements [60, 62, 81, 39, 33, 63]. TRR [81, 20, 36] sends targeted REF commands [20] (or simply ACT commands) to possible victim rows.

HammerSim allows simple extensions to the existing data structures to implement a RowHammer mitigation techniques like [44, 47, 71, 29, 98, 85]. Further, HammerSim allows tweaking such mechanisms to adhere to the timing constraints set by existing TRR mechanisms via the Inhibitor module. To show that, we implement some of the classic and *state-of-the-art* RowHammer mitigations like PARA [44] and TWICE [47].

4.1.4. Error Correcting Code. Another direction of RowHammer mitigation is ECC [13, 55, 77, 96]. ECC-enabled DIMMs are commonly available in the market

today. DDR5 DIMMs are shipped out with an on-chip proprietary ECC implementation [38, 57].

Qureshi *et al.* suggested investigating RowHammer defenses via newer ECC algorithms [77]. Online HammerSim enables functional ECC modeling to study RowHammer defenses in a full-system context. It supports selective error correction during simulation, allowing researchers to evaluate ECC schemes under realistic workloads. For demonstration, we implement a SECDED algorithm (1-bit correction, 2-bit detection) using Cojocar *et al.* reverse engineered ECC algorithm [13] on a DDR3 DIMM within HammerSim.

ECC bits are computed from a user-supplied pMatrix and applied on-demand to minimize simulation overhead. This capability allows exploration of new ECC strategies and their effectiveness against RowHammer-induced corruption.

4.1.5. RowHammer Applications. HammerSim enables researchers to explore RowHammer-based applications such as PUFs [83, 99] and DRAM fingerprinting [93, 48] by modeling cell-level variation maps in simulation. Unlike prior simulators that assume uniform vulnerability, HammerSim uses device maps derived from real DIMMs or statistical models to capture per-cell variation. This capability also supports evaluating defenses like row-swapping [80, 95, 24] by identifying strong and weak cells, reducing overhead while preserving security. Further, in Section 5.3.2, we show how HammerSim can be used to study RowHammer-based soft-errors [15].

4.2. Limitations

HammerSim introduces invasive changes to gem5’s memory model by embedding RowHammer tracking and mitigation logic directly in the memory controller interface, rather than using separate SimObjects. This design mirrors real DRAM behavior, where TRR operates on-chip. However, this approach requires periodic updates and rebasing as gem5 evolves. Making changes to the source is intuitive as our RowHammer trackers get triggered when `activateBank()` is called and the inhibitor is called at `processRefreshEvent()`, similar to the on-chip logic of a typical DIMM [20].

A key limitation is the abstraction gap: simulators [79, 49, 45] model DRAM at the logical level (banks and cache lines), whereas physical DIMMs activate entire rows spanning multiple chips. HammerSim currently applies bitflips within the same logical bank as the aggressor, while ensuring the correctness of the victim rows. This means that the data is tracked at the cache line level using simulator-dependent data objects.

Another limitation is reliance on statistically generated variation maps. While multivariate models approximate process variation and are popular [100], they may deviate from real DIMM characteristics. To address this, HammerSim supports user-supplied device maps derived from hardware experiments (e.g., Blacksmith [36], TRRespass [20]).

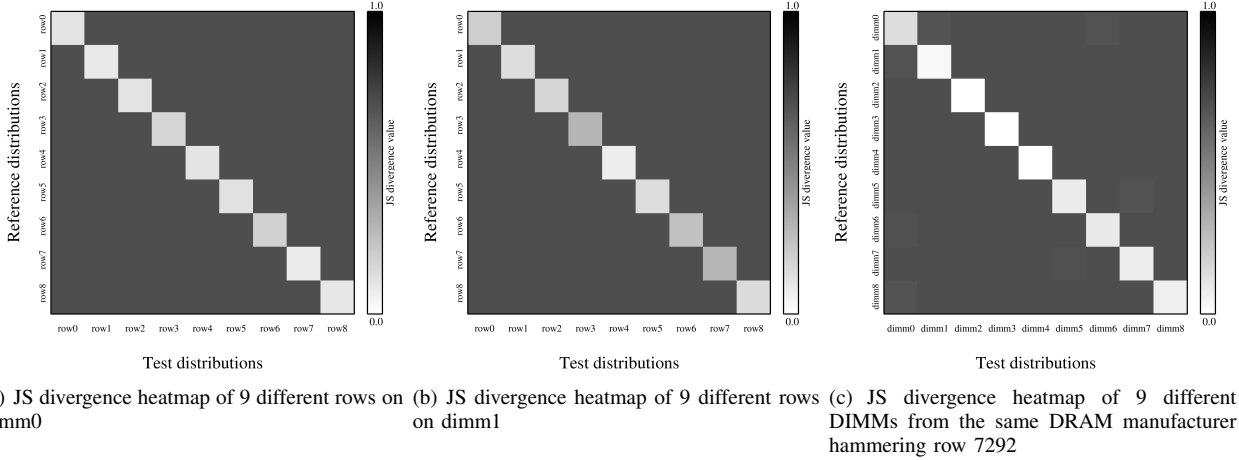


Figure 7: JS divergence heatmaps across different victim rows of the same DIMM and across multiple DIMMs on real hardware and HammerSim.

ECC is functionally implemented in HammerSim. Unlike a real ECC DIMM that stores the checksum bits on the DRAM array, we compute the checksum on the fly to model functional correctness. This allows newer ECC algorithms to be evaluated online during simulation. However, checksum data corruption and timing correctness of the memory are not modeled.

Finally, HammerSim does not model true and anti cells [44], as these mappings are proprietary. A charged DRAM capacitor does not necessarily mean a value of one [44, 36, 20]. Extending HammerSim to enable such mappings is trivial and can be implemented the same way as the device map. This does not affect the RowHammer probabilities of a bitflip.

5. Evaluation

HammerSim is implemented in gem5 [53], a full-system cycle-level simulator. We model the probability distributions of RowHammer (Section 3) within gem5’s memory interface.

HammerSim allows users to perform several kinds of experiments. We narrow down the search space and perform three case studies to evaluate HammerSim. We fix our experimental system as a commodity-class system in most of our experiments. The parameters of the system are described in Table 2.

We validate our simulation model against real hardware in Section 5.1. In Section 5.2, we show how a full system RowHammer application can be studied in HammerSim and how ECC can functionally correct bitflips. Finally, we also demonstrate how benign applications are affected by RowHammer as the RowHammer threshold is decreasing with each DRAM generation.

TABLE 2: Simulated System Parameters

Parameter	Specification
CPU type	TimingSimpleCPU
CPU core count	1/2/8
L1 cache per core	32 KiB + 32 KiB (I + D)
L2 cache	256 KiB
DRAM technology	DDR3/DDR4
DRAM size	1/2 GB

5.1. Case Study I: Validation

We want HammerSim to model existing DIMMs inside the simulator. This implies that we also need to model TRRs installed in the DIMMs. Towards this, we have implemented RNG-based TRR mitigation techniques in HammerSim, conceptually similar to U-TRR [30]. We use Blacksmith [36]’s fuzzer to find access patterns on the same DIMMs. Bitflips are generated per bank. We record the aggressor traffic. We encode the bitflip maps generated from the hardware and plug in our simulation as the variation map.

Our goal in this case study is to model a similar bitflip map in simulation. We split the test cases into within rows of a single DIMM and within banks of multiple DIMMs. Blacksmith’s fuzzer bypasses TRR with a specific memory access pattern. We use gem5’s traffic generator to generate synthetic traffic for the aggressor rows, recreating the attack. gem5’s traffic generator can send a large amount of traffic at the minimum interval of 1 ps. We adjust our traffic pattern to mimic real systems with caches and tune the RowHammer probabilities to account for such high traffic rates. Note that the device address mapping scheme, *i.e.*, physical to the DRAM device mapping, is different in the experimental system and the simulated system. We make sure that the reference and the test systems have compatible mapping.

We compare the weak columns of hardware-generated bitflips with the simulated ones for single rows. We use JS

Divergence [65] as the similarity metric to compare bitflip maps. JS divergence is a method of measuring the similarity between two probability distributions. A lower JS divergence value means the two distributions are more similar. We present our results as a heatmap of JS divergence values with references collected from the hardware and tested using HammerSim. We select 9 rows from two different DIMMs from the same DRAM manufacturer and show our findings in Figure 7(a) and Figure 7(b). We see smaller JS divergence values for the same reference and test case, suggesting a closer to hardware model of RowHammer in HammerSim.

Next, we repeat the same experiment across multiple DIMMs. We select 9 different DIMMs from the same manufacturer and model the RowHammer device map in HammerSim. We plot the heatmap of JS divergence values in Figure 7(c), which shows a close resemblance to the real hardware.

Takeaway 1: HammerSim allows localized RowHammer bitflip simulation via the device map. Weak cells per row can be modeled with high fidelity in HammerSim, up to the capacitor level, given that TRR does not interfere. While a correct reverse-engineered TRR implementation is not the goal of this paper, we show how HammerSim can be used to model the entire DIMM in simulation for higher correctness.

5.2. Case Study II: Full-System Applications and Data Corruption

We show the execution of Google’s `rowhammer-test` [84] inside the simulator in a full-system setting. The OS abstractions are bypassed by `rowhammer-test`, as it can perform both single-sided and double-sided RowHammer attacks. Our objective is to test whether we can use `rowhammer-test` as a standard RowHammer benchmark workload.

We simulated `rowhammer-test` inside HammerSim using a reproducible `gem5-resource` [11] disk-image. We hammered the memory using the single-sided RowHammer attacks from `rowhammer-test` on a system specified in Table 2. We ran a simulation of one iteration of `rowhammer-test`. We show the bitflip count in Figure 8 by varying the probability of a bitflip. `rowhammer-test`’s

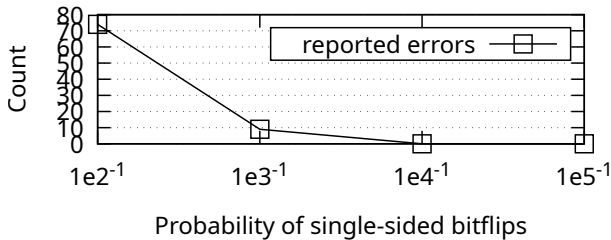


Figure 8: Reported bit errors by the simulator.

output is printed on `stdout` with the number of bitflips and the change in data, as shown in Figure 9.

```
Starting gem5 init... trying to read run script file via readfile.
Running script from gem5-bridge stored in /tmp/script
rowhammer_test
[sudo] password for gem5: clear
cloud-init[1290]: Cloud-init v. 23.1.2-0ubuntu0-22.04.1 finished at Tue,
source DataSourceNone. Up 43.50 seconds
cloud-init[1290]: 2025-12-02 18:47:57,632 - cc_final_message.py[WARNING]
Iteration 0 (after 0.01s)
  Took 338.2 ms per address set
  Took 3.38249 sec in total for 10 address sets
  Took 78.298 nanosec per memory access (for 43200000 memory accesses)
  This gives 102173 accesses per address per 64 ms refresh period
error at 0x7fae5ab6d1f0: got 0xffffffffffffffef
error at 0x7fae5bdc31d0: got 0xffffffffffffffef
error at 0x7fae5bdc31d0: got 0xffffffffffffffef
error at 0x7fae5bdc3200: got 0xffffffffffffffef
error at 0x7fae5bdc32a8: got 0xffffffffffffffef
error at 0x7fae5bdc33c8: got 0xffffffffffffffef
error at 0x7fae5e2cf008: got 0xffffffffffffffef
```

Figure 9: `stdout` of `rowhammer-test` when the *single-sided* RowHammer attack was performed online using HammerSim.

As mentioned in Section 4.2, we do not model true and anti-DRAM cells. For a bitflip, we flip the contents of the capacitor regardless of a charged or an unchanged capacitor. `rowhammer-test` writes `0xFF` as the data pattern and compares it for bitflips later. Bitflips are XOR on the exact capacitor location.

Takeaway 2: HammerSim enables simulated data corruption shown using `rowhammer-test`. This allows system-level RowHammer attacks, defenses, and applications to be studied in simulation.

We extend the previous case study to test ECC within the simulator. We functionally model ECC, that does not account for timing changes in the DRAM. At a bitflip, we store the original data for ECC parity bit computation. When victim rows are later accessed, we call the ECC algorithm to keep the wall clock time of the simulation low. Whenever RowHammer triggers a bitflip, we compute the `gMatrix` based on an 8-byte word and the user-supplied `pMatrix`.

For single-bit errors on an 8-byte word, we use the ECC bits to correct the data when the victim row is accessed. HammerSim reports the bitflips as an output statistic. We keep the simulation parameters the same as before. As we increase the probability of a bitflip, we see multiple bitflips on the same row (Table 3). The SECDED ECC corrects up

TABLE 3: SECDED ECC statistics of correcting RowHammer-induced bitflips as reported by HammerSim. Bitflips are generated randomly based on the user-supplied probability.

Probability	Bitflips	Single Errors Corrected	Double Errors Detected	Uncorrectable Errors	Notes
1e1 ⁻¹	237	29	2	17	Kernel panic
1e2 ⁻¹	115	3	0	1	None
1e3 ⁻¹	95	1	0	0	None
1e4 ⁻¹	0	0	0	0	None

to one bit flip and detects up to 2 bit flips [13].

Takeaway 3: Data corruption in HammerSim naturally allows the simulator to investigate newer and stronger error correction codes (ECC).

5.3. Case Study III: Benign Applications

5.3.1. Offline Analysis. To demonstrate HammerSim’s offline analysis, we dump the memory access traces of two popular benchmark suites: NAS-Parallel benchmarks [4] and GAPBS [6]. In our experiment, we did not see any RowHammer access patterns in these benign applications with the standard bitflip probability parameters. With shrinking process size, the RowHammer threshold is also decreasing [20, 44]. Therefore, we perform an offline estimation of bitflips on these memory traces.

HammerSim allows us to tune probabilities of bitflips to correct over/under estimations via the post-processing tool. We show this by varying the probability of observing a single-sided RowHammer-induced bitflip from 10^{-9} to 1. The results obtained are depicted in Figure 10(a) and Figure 10(b).

The memory access pattern for NPB and GAPBS never had double-sided RowHammer patterns. First, we experiment assuming a uniform distribution of observing bitflips and did not rely on the device map. We see that there are 20041.16 instances on average across the two benchmark suites, where rows cross the DDR4 threshold of 45K ACTs.

We repeat the same evaluation using a statistically generated device map for RowHammer bitflip distribution. The idea is to ignore *strong* DRAM rows, which are resistant to RowHammer attacks. We see that on average, there is a reduction of 90.1% probability of observing a bitflip compared to the previous set of results. Our observation inside a simulator is consistent with bitflip patterns observed in prior works [93, 92, 83, 21, 48], where authors of the respective papers report localized RowHammer-induced bitflips.

Takeaway 4: This experiment shows that HammerSim can be used to investigate RowHammer access patterns at a finer granularity while enabling users to estimate RowHammer bitflip probability.

5.3.2. Online Analysis and Data Corruption Study. In our experiments, we did not see any instances of RowHammer bitflips with the default RowHammer parameters of a DDR4 DRAM DIMM. We repeated the experiment with a lower RowHammer threshold (1000) and a higher RowHammer probability ($1e1^{-1}$). However, we are still interested to observe soft errors in benign applications. Towards this, we repurpose HammerSim’s infrastructure to study RowHammer-induced bitflips in the neighborhood of the aggressor row

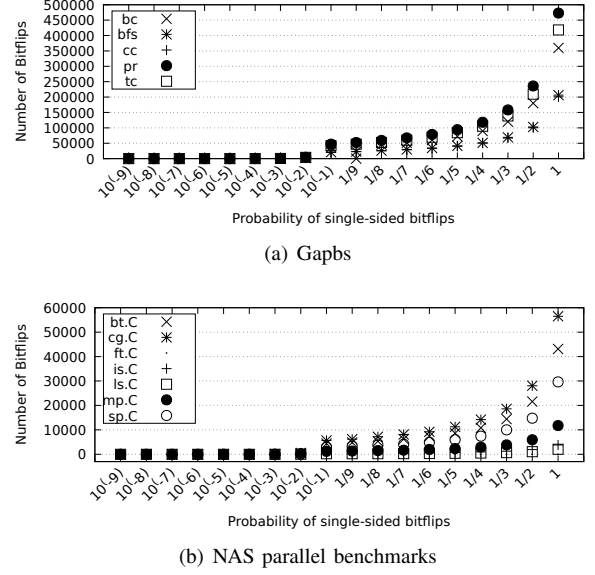


Figure 10: Offline estimation of bitflips by varying the probability of a bitflip for a DDR4 DRAM device. Note that the x-axis in both of these graphs are discontinuous as it progresses in factors of 10 until (8, 0), and then it progresses in steps of 0.1.

with a very low RowHammer threshold in benign applications. The goal is to observe silent error propagations on benign applications when the RowHammer threshold becomes dangerously low for future shrinking process.

We selected smaller workload sizes from our two selected benchmark suites (NPB class A and GAPBS size -g 20). The goal is to finish at least one iteration of ROI and validate the results on a system without caches to maximize memory accesses. We set up the experiment with data corruption enabled, set the RowHammer threshold (1), and the probability to study soft errors ($1e^{-1}$).

We model a system with a single CPU core without caches and 1 GiB DDR4 DRAM memory. For the baseline, we simulate an ideal DDR4 DRAM device without the possibility of any bitflips to measure the host system’s execution time, total memory consumption, and output correctness. We repeat this experiment by replacing the DRAM with RowHammer-affected ones and simulate the same workload with and without data corruption.

Table 4 shows the simulation statistics of HammerSim and baseline gem5. Overall, we saw HammerSim takes 10.9% and 29.2% additional wall clock time on average to simulate RowHammer DIMMs compared to a baseline unmodified gem5 for NPB and GAPBS, respectively. To maintain the additional data structures to simulate RowHammer, HammerSim consumes 50.6% and 65% additional host memory for NPB and GAPBS, respectively.

Table 4 also shows the outcome of the experiments, *i.e.*, errors induced via RowHammer. While every workload had bitflips (column 6), not all workloads failed to reach

Benchmark	Host Runtime (s)		Host Memory (Bytes)		Status	Notes
	base	ours	base	ours		
bt	1459137.85	1582481.62	9059028	13109876	✓	Silent errors
cg	19348.39	23165.54	1980816	3188016	✗	Seg. Fault
ep	162094.28	173420.81	2585996	3807536	✓	Silent errors
ft	70689.88	77007.16	2489744	3709232	✓	Silent errors
is	7786.01	9343.82	1952144	3166508	✗	Seg. Fault
lu	703200.95	703200.95	2794892	4004140	✓	Silent errors
mg	26627.67	30435.92	1975696	3624236	✓	Silent errors
sp	641277.22	755053.48	2785680	4000048	✓	Silent errors
bc	5277.73	5064.94	1878412	3091760	✓	Silent errors
bfs	254.57	413.62	1521036	2733360	✓	Silent errors
cc	399.36	566.8	1503632	2711856	✓	Silent errors
pr	6303.33	8070.86	1874316	3092784	✓	Silent errors
tc	156894.62	204465.25	2553228	3774764	✗	Seg. Fault

TABLE 4: This table shows the simulation statistics of baseline gem5 and HammerSim. We also show the data corruption status (verification of the program status) of benign applications.

the end of ROI. This means that there were silent errors that propagated to the end of the program. We did validate the results at the end of the ROI. We saw that workloads like *mg* and *ft* successfully validated their results, which means that RowHammer bitflips did not affect the outcome of the workload. This has been shown in prior research as well [15]. We plan on investigating this aspect of fault-tolerant workload behavior in the future.

Takeaway 5: While HammerSim is 11.9% slower than gem5 overall, it allows the study of soft errors on full-system applications.

5.4. Additional Results

HammerSim allows us to use statistically generated device maps. This generalizes the usability of the tool since we can use generic device maps based on some process variation-based probability distribution. Visualized results of the simulation on a specific row with 8192 columns can be seen in Figures 11(a) - 11(e). The hardware counterpart of the actual weak cell map is shown in Figure 4. The measured similarity index between the real and the simulated run is 0.31 in terms of JS Divergence [65] value.

6. Related Works

RowHammer is widely studied using trace-based simulators [87, 47, 98, 71, 7]. The major limitation of this

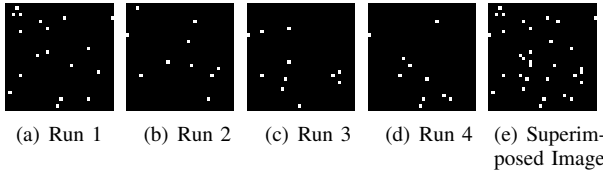


Figure 11: Simulated RowHammer result of bitflips on a single row. This set is generated using a variation map taken from an actual DRAM DIMM’s row (1278) (Figure 4)

approach is that it does not consider any of the OS effects or influences while studying both RowHammer attacks and defenses. DRAM DIMMs strictly operate based on timing. This led to the hypothesis that RowHammer mitigations or *inhibitor* sends ACT commands to possible victim rows during tREFI time, when the device is locked for normal reads and writes [20, 36, 30]. This further implies that adding victim ACTs whenever a RowHammer threshold is crossed may not be practical to implement by DRAM vendors.

gem5 has been used to study architectural and micro-architectural vulnerabilities in the past [16, 52]. There have been attempts on implementing RowHammer at a system-level in the past using gem5 [17, 18, 90]. Most of these works use an external trace-based simulator like ramulator [45] or DRAMsim3 [49] as the backend memory. France *et al.* [17] use ramulator [45] to simulate the backend memory and bitflips. The authors also did a vulnerability assessment of the RowHammer attack [18] using a similar setup. Hammulator [90] uses DRAMsim3 [49] as the backend memory.

While the aforementioned approaches are similar to HammerSim, our technique adds the ability to experiment with different probability distributions. This allows further exploration of implications of RowHammer in terms of applications and experiment system-level RowHammer mitigations. This allows researchers to perform system-level exploits and mitigations while taking different probability distributions into account.

Data corruption at a system-level allows researchers to deterministically perform RowHammer attacks and exploit system-sensitive data. This was demonstrated in rowhammer-test [84] where the authors gained access to the page table from userspace by correctly flipping the access bits. HammerSim improves upon these aforementioned works by adding uniqueness to the simulated DIMMs to represent real-world DIMMs via a device map. This not only limits HammerSim to be a RowHammer simulator, but also do DRAM/memory variation studies using simulators, making them closer to its hardware counterpart [22] at a system-level.

7. Conclusion

In this work, we introduced HammerSim, a system-level framework for modeling RowHammer within the gem5 simulator. HammerSim enables researchers to explore diverse RowHammer access patterns under realistic full-system conditions, including OS and cache interactions. By incorporating device maps, the tool captures per-DIMM vulnerability characteristics, allowing fine-grained analysis of bit-flip patterns and their implications. Our framework supports both synthetic attack traces and real RowHammer programs, enabling comprehensive evaluation of mitigation strategies. While the mitigations demonstrated in this paper are intentionally simple, HammerSim provides an extensible foundation for prototyping and benchmarking advanced defenses. Looking ahead, we plan to enhance HammerSim with richer

mitigation models and introduce a quantitative RowHammer vulnerability metric at the system level, fostering rigorous and reproducible research in DRAM reliability and security.

References

- [1] B. Aichinger, “Ddr memory errors caused by row hammer,” in *2015 IEEE High Performance Extreme Computing Conference (HPEC)*, 2015, pp. 1–5.
- [2] Z. Al-Ars, “Dram fault analysis and test generation,” 2005.
- [3] Z. B. Aweke, S. F. Yitbarek, R. Qiao, R. Das, M. Hicks, Y. Oren, and T. Austin, “Anvil: Software-based protection against next-generation rowhammer attacks,” in *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’16. New York, NY, USA: Association for Computing Machinery, 2016. [Online]. Available: <https://doi.org/10.1145/2872362.2872390>
- [4] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber *et al.*, “The NAS Parallel Benchmarks,” *The International Journal of Supercomputing Applications*, vol. 5, no. 3, pp. 63–73, 1991.
- [5] K. S. Bains and J. B. Halbert, “Distributed row hammer tracking,” 2012, US Patent US20140095780A1.
- [6] S. Beamer, K. Asanović, and D. Patterson, “The GAP Benchmark Suite,” *arXiv preprint arXiv:1508.03619*, 2015.
- [7] T. Bennett, S. Saroiu, A. Wolman, and L. Cojocar, “Panopticon: A complete in-dram rowhammer mitigation,” in *Workshop on DRAM Security (DRAMSec)*, vol. 22, 2021, p. 110.
- [8] S. Bhattacharya and D. Mukhopadhyay, “Curious case of rowhammer: Flipping secret exponent bits using timing analysis,” *Cryptology ePrint Archive*, Paper 2016/618, 2016. [Online]. Available: <https://eprint.iacr.org/2016/618>
- [9] S. Borkar, “Designing reliable systems from unreliable components: the challenges of transistor variability and degradation,” *IEEE Micro*, vol. 25, no. 6, pp. 10–16, 2005.
- [10] E. Bosman, K. Razavi, H. Bos, and C. Giuffrida, “Dedup est machina: Memory deduplication as an advanced exploitation vector,” in *2016 IEEE Symposium on Security and Privacy (SP)*, 2016, pp. 987–1004.
- [11] B. R. Bruce, A. Akram, H. Nguyen, K. Roarty, M. Samani, M. Friboz, T. Reddy, M. D. Sinclair, and J. Lowe-Power, “Enabling reproducible and agile full-system simulation,” in *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2021, pp. 183–193.
- [12] L. Cojocar, J. Kim, M. Patel, L. Tsai, S. Saroiu, A. Wolman, and O. Mutlu, “Are we susceptible to rowhammer? an end-to-end methodology for cloud providers,” in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 712–728.
- [13] L. Cojocar, K. Razavi, C. Giuffrida, and H. Bos, “Exploiting correcting codes: On the effectiveness of ecc memory against rowhammer attacks,” in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 55–71.
- [14] F. de Ridder, P. Frigo, E. Vannacci, H. Bos, C. Giuffrida, and K. Razavi, “SMASH: Synchronized many-sided rowhammer attacks from JavaScript,” in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 1001–1018. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/ridder>
- [15] F. de Ridder, P. Jattke, and K. Razavi, “Softhammer: Exploiting rowhammer bit flips without crashing,” in *5th Workshop on DRAM Security (DRAMSec)*. ETH Zurich, 2025.
- [16] Q. Forcioli, J.-L. Danger, and S. Chaudhuri, “A gem5 based platform for micro-architectural security analysis,” in *Proceedings of the 12th International Workshop on Hardware and Architectural Support for Security and Privacy*, ser. HASP ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 91–99. [Online]. Available: <https://doi.org/10.1145/3623652.3623674>
- [17] L. France, F. Bruguier, M. Mushtaq, D. Novo, and P. Benoit, “Implementing rowhammer memory corruption in the gem5 simulator,” in *2021 IEEE International Workshop on Rapid System Prototyping (RSP)*. IEEE, 2021, pp. 36–42.
- [18] L. France, M. Mushtaq, F. Bruguier, D. Novo, and P. Benoit, “Vulnerability assessment of the rowhammer attack using machine learning and the gem5 simulator - work in progress,” *Proceedings of the 2021 ACM Workshop on Secure and Trustworthy Cyber-Physical Systems*, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:233384386>
- [19] P. Frigo, C. Giuffrida, H. Bos, and K. Razavi, “Grand pwning unit: Accelerating microarchitectural attacks with the gpu,” in *2018 IEEE Symposium on Security and Privacy (SP)*, 2018, pp. 195–210.
- [20] P. Frigo, E. Vannacci, H. Hassan, V. van der Veen, O. Mutlu, C. Giuffrida, H. Bos, and K. Razavi, “Trespass: Exploiting the many sides of target row refresh,” 2020. [Online]. Available: <https://arxiv.org/abs/2004.01807>
- [21] L. Gerlach, F. Thomas, R. Pietsch, and M. Schwarz, “A rowhammer reproduction study using the blacksmith fuzzer,” in *ESORICS*, 2023.
- [22] S. Ghose, A. G. Yaglikçi, R. Gupta, D. Lee, K. Kudrolli, W. X. Liu, H. Hassan, K. K. Chang, N. Chatterjee, A. Agrawal, M. O’Connor, and O. Mutlu, “What your dram power models are not telling you: Lessons from a detailed experimental study,” *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 2, no. 3, dec 2018. [Online]. Available: <https://doi.org/10.1145/3224419>
- [23] Google LLC, ““Half-Double”: Next-Row-Over Assisted Rowhammer,” 2021. [Online]. Available: https://raw.githubusercontent.com/google/hammer-kit/main/20210525_half_double.pdf
- [24] K. Goswami, D. S. Banerjee, and S. Das, “Towards enhanced system efficiency while mitigating row hammer,” *ACM Trans. Archit. Code Optim.*, vol. 18, no. 4, Jul. 2021. [Online]. Available: <https://doi.org/10.1145/3458749>
- [25] K. Goswami, S. Das, S. Satapathy, and D. S. Banerjee, “A case for amplifying row hammer attacks via cell-coupling in dram devices,” in *Proceedings of the 2022 International Symposium on Memory Systems*, ser. MEMSYS ’22. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3565053.3565056>
- [26] D. Gruss, M. Lipp, M. Schwarz, D. Genkin, J. Juffinger, S. O’Connell, W. Schoecl, and Y. Yarom, “Another flip in the wall of rowhammer defenses,” 2018. [Online]. Available: <https://arxiv.org/abs/1710.00551>
- [27] D. Gruss, C. Maurice, and S. Mangard, “Rowhammer.js: A remote software-induced fault attack in javascript,” in *Detection of Intrusions and Malware, and Vulnerability Assessment: 13th International Conference, DIMVA 2016, San Sebastián, Spain, July 7-8, 2016, Proceedings 13*. Springer, 2016, pp. 300–321.
- [28] A. Hansson, N. Agarwal, A. Kolli, T. Wensich, and A. N. Udupi, “Simulating dram controllers for future system architecture exploration,” in *2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2014, pp. 201–210.
- [29] H. Hassan, M. Patel, J. S. Kim, A. G. Yaglikçi, N. Vijaykumar, N. M. Ghiasi, S. Ghose, and O. Mutlu, “Crow: A low-cost substrate for improving dram performance, energy efficiency, and reliability,” in *Proceedings of the 46th international symposium on computer architecture*, 2019, pp. 129–142.
- [30] H. Hassan, Y. C. Tugrul, J. S. Kim, V. van der Veen, K. Razavi, and O. Mutlu, “Uncovering in-dram rowhammer protection mechanisms: A new methodology, custom rowhammer patterns, and implications,” 2021. [Online]. Available: <https://arxiv.org/abs/2110.10603>

- [31] H. Hassan, N. Vijaykumar, S. Khan, S. Ghose, K. Chang, G. Pekhimenko, D. Lee, O. Ergin, and O. Mutlu, "Softmc: A flexible and practical open-source infrastructure for enabling experimental dram studies," in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2017, pp. 241–252.
- [32] Y. Hu, N. Brown, Y. Chen, J. Bakita, T. Chen, D. Genkin, and A. Kwong, "Gddrhammer: Greatly disturbing dram rows—cross-component rowhammer attacks from modern gpus," in *Proceedings of the 2026 IEEE Symposium on Security and Privacy (SP)*. San Francisco, CA, USA: IEEE, 2026.
- [33] Hynix Semiconductor, "Datasheet for 1Gb (32Mx32) GDDR5 SGRAM H5GQ1H24AFR," Tech. Rep. H5GQ1H24AFR, 2009.
- [34] B. Jacob, S. Ng, and D. Wang, *Memory Systems: Cache, DRAM, Disk*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2007.
- [35] Y. Jang, J. Lee, S. Lee, and T. Kim, "Sgx-bomb: Locking down the processor via rowhammer attack," in *Proceedings of the 2nd Workshop on System Software for Trusted Execution*, ser. SysTEX'17. New York, NY, USA: Association for Computing Machinery, 2017. [Online]. Available: <https://doi.org/10.1145/3152701.3152709>
- [36] P. Jattke, V. Van Der Veen, P. Frigo, S. Gunter, and K. Razavi, "Blacksmith: Scalable rowhammering in the frequency domain," in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 716–734.
- [37] P. Jattke, V. van der Veen, P. Frigo, S. Gunter, and K. Razavi, "BLACKSMITH: Rowhammering in the Frequency Domain," 2022-05, <https://github.com/comsec-group/blacksmith>.
- [38] P. Jattke, M. Wipfli, F. Solt, M. Marazzi, M. Bölskei, and K. Razavi, "ZenHammer: Rowhammer attacks on AMD zen-based platforms," in *33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, Aug. 2024, pp. 1615–1633. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/jattke>
- [39] JEDEC, "DDR5 SDRAM," Tech. Rep. JESD79-5B, August 2022.
- [40] Y. Jiang, H. Zhu, D. Sullivan, X. Guo, X. Zhang, and Y. Jin, "Quantifying rowhammer vulnerability for dram security," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 73–78.
- [41] I. Kang, W. Wang, J. Kim, S. van Schaik, Y. Tobah, D. Genkin, A. Kwong, and Y. Yarom, "{SledgeHammer}: Amplifying rowhammer via bank-level parallelism," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1597–1614.
- [42] M. N. I. Khan and S. Ghosh, "Analysis of row hammer attack on stram," in *2018 IEEE 36th International Conference on Computer Design (ICCD)*, 2018, pp. 75–82.
- [43] J. S. Kim, M. Patel, A. G. Yağlıkcı, H. Hassan, R. Azizi, L. Orosa, and O. Mutlu, "Revisiting rowhammer: An experimental analysis of modern dram devices and mitigation techniques," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 638–651.
- [44] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, "Flipping bits in memory without accessing them: An experimental study of dram disturbance errors," in *Proceeding of the 41st Annual International Symposium on Computer Architecture*. IEEE Press, 2014.
- [45] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 45–49, 2016.
- [46] A. Kogler, J. Juffinger, S. Qazi, Y. Kim, M. Lipp, N. Boichat, E. Shiu, M. Nissler, and D. Gruss, "Half-Double: Hammering from the next row over," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 3807–3824. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/kogler-half-double>
- [47] E. Lee, I. Kang, S. Lee, G. E. Suh, and J. H. Ahn, "TWiCe: Preventing Row-Hammering by Exploiting Time Window Counters," in *Proceedings of the 46th International Symposium on Computer Architecture*, ser. ISCA '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 385–396. [Online]. Available: <https://doi.org/10.1145/3307650.3322232>
- [48] D. Li, D. Liu, Y. Ren, Z. Wang, Y. Sun, Z. Guan, Q. Wu, and J. Liu, "Fphammer: A device identification framework based on dram fingerprinting," in *2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2023, pp. 1031–1040.
- [49] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.
- [50] C. S. Lin, J. Qu, and G. Saileshwar, "GPUHammer: Rowhammer attacks on GPU memories are practical," in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 5719–5738. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/lin-shaopeng>
- [51] K. Loughlin, S. Saroiu, A. Wolman, Y. A. Manerkar, and B. Kasikci, "Moesi-prime: Preventing coherence-induced hammering in commodity workloads," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 670–684. [Online]. Available: <https://doi.org/10.1145/3470496.3527427>
- [52] J. Lowe-Power, "Visualizing spectre with gem5," June 2018. [Online]. Available: <http://www.lowepower.com/jason/visualizing-spectre-with-gem5.html>
- [53] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, B. Beckmann, S. Bharadwaj, G. Black, G. Bloom, B. R. Bruce, D. R. Carvalho, J. Castrillon, L. Chen, N. Derumigny, S. Diestelhorst, W. Elsasser, C. Escuin, M. Fariborz, A. Farmahini-Farahani, P. Fotouhi, R. Gambord, J. Gandhi, D. Gope, T. Grass, A. Gutierrez, B. Hanindhito, A. Hansson, S. Haria, A. Harris, T. Hayes, A. Herrera, M. Horsnell, S. A. R. Jafri, R. Jagtap, H. Jang, R. Jeyapaul, T. M. Jones, M. Jung, S. Kannoth, H. Khaleghzadeh, Y. Kodama, T. Krishna, T. Marinelli, C. Menard, A. Mondelli, M. Moreto, T. Mück, O. Naji, K. Nathella, H. Nguyen, N. Nikoleris, L. E. Olson, M. Orr, B. Pham, P. Prieto, T. Reddy, A. Roelke, M. Samani, A. Sandberg, J. Setoain, B. Shingarov, M. D. Sinclair, T. Ta, R. Thakur, G. Travaglini, M. Upton, N. Vaish, I. Vougioukas, W. Wang, Z. Wang, N. Wehn, C. Weis, D. A. Wood, H. Yoon, and Éder F. Zulian, "The gem5 simulator: Version 20.0+," 2020. [Online]. Available: <https://arxiv.org/abs/2007.03152>
- [54] H. Luo, A. Olgun, A. G. Yağlıkcı, Y. C. Tuğrul, S. Rhyner, M. B. Cavlak, J. Lindegger, M. Sadrosadati, and O. Mutlu, "Rowpress: Amplifying read disturbance in modern dram chips," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–18.
- [55] E. Manzhosov and S. Sethumadhavan, "Polymorphic error correction," in *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '24. IEEE Press, 2024, p. 246–262. [Online]. Available: <https://doi.org/10.1109/MICRO61859.2024.00027>
- [56] M. Marazzi and K. Razavi, "Risc-h: Rowhammer attacks on risc-v," in *4th Workshop on DRAM Security (DRAMSec) co-located with ISCA 2024*. ETH Zurich, 2024.
- [57] D. Meyer, P. Jattke, M. Marazzi, S. Qazi, D. Moghimi, and K. Razavi, "Phoenix: Rowhammer attacks on ddr5 with self-correcting synchronization," 2026.
- [58] Micron Technology, "TN-46-12: Mobile DRAM Power-Saving Features and Power Calculations," Tech. Rep. TN46_12, 2005.
- [59] —, "DDR2 SDRAM," Tech. Rep. MT47H512M4, MT47H256M8, MT47H128M16, 2006.

- [60] —, “TN-41-01: Calculating Memory System Power for DDR3,” Tech. Rep. TN41_01DDR3, January 2007.
- [61] —, “1.35V DDR3L SDRAM SODIMM,” Tech. Rep. MT16KTF51264HZ, MT16KTF1G64HZ, 2011.
- [62] Micron Technology, “DDR4 SDRAM,” Tech. Rep. MT40A2G4, MT40A1G8, MT40A512M16, 2015.
- [63] Micron Technology, “TN-ED-03: GDDR6: The Next-Generation Graphics DRAM,” Tech. Rep. TN-ED-03: GDDR6, 2017.
- [64] O. Mutlu and J. S. Kim, “Rowhammer: A retrospective,” *Trans. Comp.-Aided Des. Integ. Cir. Sys.*, vol. 39, no. 8, p. 1555–1571, aug 2020. [Online]. Available: <https://doi.org/10.1109/TCAD.2019.2915318>
- [65] Notes on AI, “Jensen-Shannon Divergence,” <https://notesonai.com/Jensen%E2%80%93Shannon+Divergence>.
- [66] A. Olgun, H. Hassan, A. G. Yaglikci, Y. C. Tugrul, L. Orosa, H. Luo, M. Patel, O. Ergin, and O. Mutlu, “DRAM Bender: An Extensible and Versatile FPGA-based Infrastructure to Easily Test State-of-the-art DRAM Chips,” *IEEE TCAD*, 2023.
- [67] A. Olgun, Y. C. Tugrul, N. Bostanci, I. E. Yuksel, H. Luo, S. Rhyner, A. G. Yaglikci, G. F. Oliveira, and O. Mutlu, “Abacus: All-bank activation counters for scalable and low overhead rowhammer mitigation,” 2023.
- [68] L. Orosa, U. Rührmair, A. G. Yaglikci, H. Luo, A. Olgun, P. Jatkke, M. Patel, J. Kim, K. Razavi, and O. Mutlu, “Spyhammer: Using rowhammer to remotely spy on temperature,” *arXiv preprint arXiv:2210.04084*, 2022.
- [69] L. Orosa, A. G. Yaglikci, H. Luo, A. Olgun, J. Park, H. Hassan, M. Patel, J. S. Kim, and O. Mutlu, “A deeper look into rowhammer’s sensitivities: Experimental analysis of real dram chips and implications on future attacks and defenses,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 1182–1197. [Online]. Available: <https://doi.org/10.1145/3466752.3480069>
- [70] K. Park, D. Yun, and S. Baeg, “Statistical distributions of rowhammering induced failures in ddr3 components,” *Microelectronics Reliability*, vol. 67, pp. 143–149, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0026271416304000>
- [71] Y. Park, W. Kwon, E. Lee, T. J. Ham, J. Ho Ahn, and J. W. Lee, “Graphene: Strong yet lightweight row hammer protection,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 1–13.
- [72] D. A. Patterson and J. L. Hennessy, *Computer Architecture: A Quantitative Approach*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1990.
- [73] P. Pessl, D. Gruss, C. Maurice, M. Schwarz, and S. Mangard, “DRAMA: Exploiting DRAM addressing for Cross-CPU attacks,” in *25th USENIX Security Symposium (USENIX Security 16)*. Austin, TX: USENIX Association, Aug. 2016, pp. 565–581. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/pessl>
- [74] A. Plin, F. Fauberteau, and N. Nguyen, “Opengl gpu-based rowhammer attack (work in progress),” in *European Symposium on Research in Computer Security*. Springer, 2025, pp. 347–358.
- [75] M. Poremba and Y. Xie, “Nvmain: An architectural-level main memory simulator for emerging non-volatile memories,” in *2012 IEEE Computer Society Annual Symposium on VLSI*, 2012, pp. 392–397.
- [76] J. Power, J. Hestness, M. S. Orr, M. D. Hill, and D. A. Wood, “gem5-gpu: A heterogeneous cpu-gpu simulator,” *IEEE Computer Architecture Letters*, vol. 14, no. 1, pp. 34–36, 2014.
- [77] M. Qureshi, “Rethinking ecc in the era of row-hammer,” *DRAMSec*, 2021.
- [78] V. Ramadas, D. Koucheinia, N. Osuji, and M. D. Sinclair, “Closing the gap: Improving the accuracy of gem5’s gpu models.” 5th gem5 Users’ Workshop, 2023.
- [79] P. Rosenfeld, E. Cooper-Balis, and B. Jacob, “Dramsim2: A cycle accurate memory system simulator,” *IEEE Computer Architecture Letters*, vol. 10, no. 1, pp. 16–19, 2011.
- [80] G. Saileshwar, B. Wang, M. Qureshi, and P. J. Nair, “Randomized row-swap: mitigating row hammer by breaking spatial correlation between aggressor and victim rows,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1056–1069. [Online]. Available: <https://doi.org/10.1145/3503222.3507716>
- [81] Samsung Electronics, “DDR4 SDRAM,” Tech. Rep., 2014. [Online]. Available: https://www.samsung.com/semiconductor/global.semi/file/resource/2017/11/DDR4_Device_Operations_Rev11_Oct_14-0.pdf
- [82] S. R. Sarangi, B. Greskamp, R. Teodorescu, J. Nakano, A. Tiwari, and J. Torrellas, “Varius: A model of process variation and resulting timing errors for microarchitects,” *IEEE Transactions on Semiconductor Manufacturing*, vol. 21, no. 1, pp. 3–13, 2008.
- [83] A. Schaller, W. Xiong, N. A. Anagnostopoulos, M. U. Saleem, S. Gabmeyer, S. Katzenbeisser, and J. Szefer, “Intrinsic rowhammer PUFs: Leveraging the rowhammer effect for improved security,” in *2017 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, may 2017. [Online]. Available: <https://doi.org/10.1109%2FHst.2017.7951729>
- [84] M. Seaborn and T. Dullien, “Exploiting the dram rowhammer bug to gain kernel privileges,” *Black Hat*, vol. 15, p. 71, 2015.
- [85] S. M. Seyedzadeh, A. K. Jones, and R. Melhem, “Counter-Based Tree Structure for Row Hammering Mitigation in DRAM,” *IEEE Computer Architecture Letters*, vol. 16, no. 1, pp. 18–21, 2017. [Online]. Available: <https://doi.org/10.1109/LCA.2016.2614497>
- [86] S. M. Seyedzadeh, D. Kline, A. K. Jones, and R. Melhem, “Mitigating bitline crosstalk noise in dram memories,” in *Proceedings of the International Symposium on Memory Systems*, ser. MEMSYS ’17. New York, NY, USA: Association for Computing Machinery, 2017, p. 205–216. [Online]. Available: <https://doi.org/10.1145/3132402.3132410>
- [87] M. Son, H. Park, J. Ahn, and S. Yoo, “Making DRAM Stronger Against Row Hammering,” in *Proceedings of the 54th Annual Design Automation Conference 2017*, ser. DAC ’17. New York, NY, USA: Association for Computing Machinery, 2017. [Online]. Available: <https://doi.org/10.1145/3061639.3062281>
- [88] A. Tatar, C. Giuffrida, H. Bos, and K. Razavi, “Defeating software mitigations against rowhammer: A surgical precision hammer,” in *Research in Attacks, Intrusions, and Defenses*, M. Bailey, T. Holz, M. Stamatogiannakis, and S. Ioannidis, Eds. Cham: Springer International Publishing, 2018, pp. 47–66.
- [89] M. Technology, “Ddr4 rdimm 64gb x72 ecc module datasheet,” 2022, accessed: 2025-11-29. [Online]. Available: https://advdownload.advantech.com/productfile/PIS/96D4-64G3200ER-M2/file/96D4-64G3200ER-M2_datasheet20220926100802.pdf
- [90] F. Thomas, L. Gerlach, and M. Schwarz, “Hammulator: Simulate now-exploit later,” in *3rd Workshop on DRAM Security*, 2023.
- [91] Y. Tobah, A. Kwong, I. Kang, D. Genkin, and K. G. Shin, “Specchammer: Combining spectre and rowhammer for new speculative attacks,” in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 681–698.
- [92] V. van der Veen, Y. Fratantonio, M. Lindorfer, D. Gruss, C. Maurice, G. Vigna, H. Bos, K. Razavi, and C. Giuffrida, “Drammer: Deterministic rowhammer attacks on mobile platforms,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’16. New York, NY, USA: Association for Computing Machinery, 2016. [Online]. Available: <https://doi.org/10.1145/2976749.2978406>

- [93] H. Venugopalan, K. Goswami, Z. A. Din, J. Lowe-Power, S. T. King, and Z. Shafiq, "Centauri: Practical rowhammer fingerprinting," *arXiv preprint arXiv:2307.00143*, 2023.
- [94] D. Wang, B. Ganesh, N. Tuaycharoen, K. Baynes, A. Jaleel, and B. Jacob, "DRAMsim: A Memory System Simulator," *ACM SIGARCH Computer Architecture News*, vol. 33, no. 4, pp. 100–107, November 2005.
- [95] J. Woo, G. Saileshwar, and P. J. Nair, "Scalable and secure row-swap: Efficient and safe row hammer mitigation in memory systems," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 374–389.
- [96] S. C. Woo, W. Elsasser, M. Hamburg, E. Linstadt, M. R. Miller, T. Song, and J. Tringali, "Rampart: Rowhammer mitigation and repair for server memory systems," 2023.
- [97] X.-C. Wu, T. Sherwood, F. T. Chong, and Y. Li, "Protecting page tables from rowhammer attacks using monotonic pointers in dram true-cells," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 645–657. [Online]. Available: <https://doi.org/10.1145/3297858.3304039>
- [98] A. G. Yağlıkçı, M. Patel, J. S. Kim, R. Azizi, A. Olgun, L. Orosa, H. Hassan, J. Park, K. Kanellopoulos, T. Shahroodi, S. Ghose, and O. Mutlu, "Blockhammer: Preventing rowhammer at low cost by blacklisting rapidly-accessed dram rows," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 345–358. [Online]. Available: <https://doi.org/10.1109/HPCA51647.2021.00037>
- [99] S. Zeitouni, D. Gens, and A.-R. Sadeghi, "It's hammer time: How to attack (rowhammer-based) dram-pufs," in *Proceedings of the 55th Annual Design Automation Conference*, ser. DAC '18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: <https://doi.org/10.1145/3195970.3196065>
- [100] B. Zhao, Y. Du, J. Yang, and Y. Zhang, "Process variation-aware nonuniform cache management in a 3d die-stacked multicore processor," *IEEE Transactions on Computers*, vol. 62, no. 11, pp. 2252–2265, 2012.

Appendix

1. Navigating the Source Code

One of the objectives of this tool is to be convenient and handy to get started with. Towards this, we document and provide a detailed step-by-step instruction list.

The repository is hosted at [github²](https://github.com/darchr/gem5-rowhammer). The repository is hosted as fork of gem5. HammerSim is encompassed within the following source files of the parent gem5 repository:

- `src/mem/DRAMInterface.py` All the high-level parameters to simulate RowHammer including RowHammer threshold, TRR mechanism, RowHammer probabilities *etc.* are defined as a part of the DRAM interface. Since read/write disturbance follows a different distribution in the case of NVM devices [42], we separate these parameters out and keep it limited to the DRAM interface in gem5.
- `src/mem/mem_interface.hh` Post gem5 v.21, the memory interface has been split into two different interfaces to distinguish between DRAM and NVM devices. This file includes the necessary variables to keep a track of aggressor and victim rows.

2. <https://github.com/darchr/gem5-rowhammer>.

- `src/mem/dram_interface.hh/cc` Majority of the changes necessary to implement HammerSim is incorporated here. The latter half of this subsection discusses more on this file.
- `src/mem/SConscript`

Another way of looking at HammerSim is to simulate gem5 with a unique DRAM DIMM in each experiment. This is enabled via the `device_file` that the user supplies in as an input parameter. The `device_file` is essentially a weak cell distribution map, which we previously referred to as the device map. This map is generated directly from a DRAM DIMM using SoftMC [31] if using an FPGA, TRRespass [20], Blacksmith [36] *etc.*, if using a desktop-class machine. Alternately this map can also be statistically generated using process variation modeling techniques like VARIUS [82].

For simplicity, the map is encoded as a .json file with the format listed in Code Listing 1. This resolution of weak cells can be tuned as per users' needs. A coarse-grained variation map with weak rows can model uniform distribution of a bitflip across the row, while a column or a capacitor-level map can simulate bitflips with higher fidelity with the overhead of larger variation maps.

```

1 {
2   "rank_number": {
3     "bank_number": {
4       "row_number": [...,
5       list_of_all_weak_columns, .. ],
6     }
7   }

```

Listing 1: Device Map Format

TRRs, the only practical solution to RowHammer has to ensure that the timing constraints of the DRAM device is consistent. Therefore, a new RowHammer detection mechanism has to be incorporated while the DRAM ACTIVATES and its corresponding mitigation has to be within a REFRESH. We have implemented mitigations as `trrVariant` in the codebase. Adding a new mitigation mechanism has to be done in the `src/mem/dram_interface.cc` file. This has to be within the `trrVariant` case. The Code Listing 2 shows how to add a new RowHammer detection and mitigation mechanism in HammerSim. Code Listing 3 shows where HammerSim checks for bitflips in the codebase.

```

1 // the sampler/counter mechanism is defined here
2 void
3 DRAMInterface::activateBank(Rank& rank_ref, Bank
4   & bank_ref, Tick act_tick, uint32_t row) {
5   ...
6   switch (trrVariant) {
7     ...
8     case N: {
9       // write a new mitigation mechanism
10      // here.
11    }
12    ...
13  }
14 }

```

```

15
16 // the inhibitor mechanism is implemented here.
17 // this is because the inhibitor mechanism is
18 // triggered when the DRAM device is locked for
19 // refreshing.
20 void
21 DRAMInterface::Rank::processRefreshEvent() {
22     ...
23     switch(dram.trrVariant) {
24         ...
25         case N: {
26             // write the inhibitor mechanism
27             // here to keep DRAM timing
28             // consistent.
29         }
30         ...
31     }
32     ...
33 }

```

Listing 2: Adding a new RowHammer Mitigation in HammerSim

From previous work, we know that the method of storing data in DRAM DIMMs is proprietary to the manufacturers [36, 93]. A charged capacitor can either store a 0 or a 1. Base version of gem5 does not implement this due to obvious overhead contrains. Therefore this is a limitation of HammerSim, as mentioned in Section 4.2.

To implement data corruption, we selectively corrupt a random capacitor of a column. We assume valid data is represented as a charged capacitor in the simulation. `rowhammer-test`, HammerSim allows selective implementation of the same by only altering victim rows' data via the method `doMemoryCorruption()` thus solving the overhead issue.

```

1 void
2 DRAMInterface::checkRowHammer(Bank& bank_ref,
3     MemPacket* mem_pkt) {
4     ...
5 }

```

Listing 3: Checking for RowHammer Bitflips

2. RowHammer Simulation Parameters

Table 5 lists all the available parameters to the user for simulating RowHammer. We broadly divide the parameters into five distinct categories:

- 1) *RowHammer simulation parameters*: Parameters used to vary the probability of observing a bitflip based on a certain attack pattern. In addition, the user can input a RowHammer variation map as a JSON file as the `device_map`.
- 2) *RowHammer mitigation parameters*: We have implemented two versions of TRR from prior literature [20, 30, 36, 37]. We preserve the *sampler-inhibitor* structure [20] within the DRAM interface. We provide several parameters to study the RowHammer sampler.
- 3) *Features*: We treat memory corruption as a feature in the current version of HammerSim.
- 4) *ECC parameters*: We add a couple of ECC parameters that the user can vary. This includes the ECC algorithm

Category	Parameter	Description
RowHammer simulation parameters	<code>device_file</code>	Path to a variation map.
	<code>rowhammer_threshold</code>	Standard RowHammer bitflip threshold.
	<code>single_sided_prob</code>	Probability of observing a single-sided RowHammer bitflip.
	<code>double_sided_prob</code>	Probability of observing a double-sided RowHammer bitflip.
RowHammer mitigation parameters	<code>half_double_prob</code>	Probability of observing a half-double RowHammer bitflip.
	<code>trr_variant</code>	Mitigation variant to use during simulation.
	<code>trr_threshold</code>	Every mitigation needs to have a threshold to send refreshes.
	<code>companion_threshold</code>	Some mitigations use multiple tables. This structure addresses that.
	<code>counter_table_length</code>	The length of the mitigation table.
	<code>companion_table_length</code>	The length of the secondary counter table.
Features	<code>enable_memory_corruption</code>	Simulate live memory corruption.
ECC parameters	<code>enable_ecc</code>	Binary input to enable ECC.
	<code>p_matrix</code>	A plain text file of a <code>p_matrix</code> .
	<code>ecc_algorithm</code>	To switch between multiple ECC algorithms.
Utilities	<code>trr_stats_dump</code>	Dumps all TRR variable counts.
	<code>rh_stat_file</code>	Dumps all bitflip variable counts.
	<code>synthetic_traffic</code>	To enable synthetic traffic.

TABLE 5: All parameters available to the user to simulate RowHammer in HammerSim.

and its necessary input parameters for error correction (*e.g. pMatrix etc.*).

- 5) *Utilities*: We further add a couple of useful features to the user, such as dumping the TRR and bitflip information, mainly for offline HammerSim.